



OASIS Security Report

Insecure Direct Object Reference Security Analysis

Date: 2026-06-11 10:15:28 — **Model:** mistral:latest

Vulnerability: Insecure Direct Object Reference

Project: clean_contracts

Analysis root: `../clean`

Table of contents

▢ Summary

▢ Statistics

▢ Files

▢ Detailed analysis

▢ Assistant

△ Errors & notes

Summary

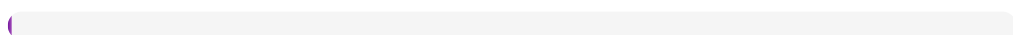
Analyzed **2** file(s) for Insecure Direct Object Reference.

[Return to table of contents](#)

Statistics

Files analyzed: 2 **Total findings:** 2

Critical



0

High		0
Medium		2
Low		0

[Return to table of contents](#)

Files

File	Similarity
<code>/work/project/dataset/clean/simple_dao_fixed.sol</code>	0.639
<code>/work/project/dataset/clean/modifier_reentrancy_fixed.sol</code>	0.617

[Return to table of contents](#)

Detailed analysis

File 1: /work/project/dataset/clean/simple_dao_fixed.sol

Similarity score: 0.639

Source lines: 1-34

► Finding 1.1: Vulnerability found (Medium)

Vulnerable code

```
credit[msg.sender]
```

Explanation

The contract exposes direct reference to an internal object (user's credit) without proper access control checks.

Impact

Unauthorized access to data or privilege escalation

Entry point

withdraw function

Execution path

```
'''  
user -> withdraw(amount) -> credit[msg.sender] -  
require(msg.sender.call.value(amount)());  
'''
```

Parameters

amount

Exploitation steps

- Call the withdraw function with an arbitrary amount
- Manipulate the credit variable directly as msg.sender is used

Conditions

Attacker has the ability to call the withdraw function.

Remediation

Implement proper access controls, use indirect references or check-effects-interactions pattern for credit[msg.sender]

Secure example

```
credit[msg.sender].withdraw(amount)
```

File 2: /work/project/dataset/clean/modifier_reentrancy_fixed.sol

Similarity score: 0.617

Source lines: 1-42

► Finding 2.1: Vulnerability found (Medium)

Vulnerable code

```
supportsToken hasNoBalance public{
    tokenBalance[msg.sender] += 20;
}
```

Explanation

The contract function airDrop uses the msg.sender address directly as the object reference, bypassing proper authorization checks.

Impact

An attacker can manipulate the balance of any contract that supports the Nu Token by calling airDrop while having a zero balance.

Entry point

airDrop function

Execution path

```

'''
+-----+
| Client | --airDrop-->| ModifierEntrancy | --tokenBalance+
+---| Bank |
+-----+
'''
,
```

Parameters

msg.sender

Exploitation steps

- Call airDrop while having a zero balance
- Manipulate the msg.sender parameter to any contract address

Remediation

Implement proper access controls or use indirect object references.

Secure example

```
airDrop() supportsToken() hasNoBalance {  
    require(keccak256(abi.encodePacked("Nu Token")) == bank.supportsToken());  
    if (tokenBalance[msg.sender] == 0) {  
        tokenBalance[msg.sender] += 20;  
    }  
}
```

[Return to table of contents](#)

Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)